

Assignment4Aparnainversetheory

November 3, 2025

```
[17]: import numpy as np
import matplotlib.pyplot as plt
from numpy.linalg import solve
#Forward model
#Define the basal velocity
x = np.linspace(0,100,1001)
u_b = 0.25 * (1.0 + np.cos(2.0 * np.pi * (x - 25.0) / 50.0)) #The same as
    ↳ defined in the synthetic glacier test in the reference paper
#Define constants
u_c = 1 #The values of constants appearing in the expression for center
    ↳ flowline velocity along with the ice thickness and slope are taken in a way
    ↳ that u_c remains constant across x and is equal to 1
dx = x[1]-x[0]
L = x[-1]
#Gaussian kernel for the model
sigma = 10.0 #can be changed as required
x_kernel = x - x.mean()
g = np.exp(-0.5 * (x_kernel / sigma)**2)
g /= np.trapz(g, x_kernel)
ln_us = np.convolve(np.log(1.0 + u_b), g, mode='same') * dx
u_s = np.exp(ln_us) #surface velocity from the forward model by putting basal
    ↳ velocity as the input
#Adding noise to the surface velocity obtained through forward model
noise = 0.01*np.random.randn(len(x))*np.mean(u_s) #adding gaussian noise , can
    ↳ be altered as desired
u_s_noisy = u_s + noise
plt.figure(figsize=(9,4))
plt.plot(x, u_b, 'k--', lw=2, label='Input Basal velocity $u_b$ in FM') #FM-
    ↳ forward model
plt.plot(x, u_s, 'b-', lw=2, label='Calculated Surface velocity $u_s$ in FM')
plt.scatter(x[:25], u_s_noisy[:25], color='red', s=15, label='Noisy data')
plt.xlabel('x')
plt.ylabel('Velocity')
plt.title('Forward Model')
plt.legend()
plt.grid(alpha=0.3)
plt.xlim(0, 100)
```

```

plt.show()
# Inverse model
# We linearize to get the inverse problem in the form  $d=Gm$  where  $d$  is the data
↳points - the noisy surface velocity generated earlier
#  $m$  is the linearized operator  $\ln(1+u_b) \sim \ln(u_b)$ 
#  $G$  is the martrix form of the discretized convolution
d = np.log(np.maximum(u_s_noisy, 1e-8)) #To avoid log0
# Forward operator for differentiating  $m$  to perform first order Tikonov
↳regularization
N = x.size
D = np.zeros((N-1,N))
for i in range(N-1):
    D[i, i] = -1.0 / dx
    D[i, i+1] = 1.0 / dx
#Define regularization parameter
lam = 0.5
#  $G$  matrix is defined as  $G[i,j] = g(x_j - x_i) * dx$ 
G = np.zeros((N, N))
kernel_func = lambda d: np.exp(-0.5 * (d/sigma)**2)
for i, xi in enumerate(x):
    gvals = kernel_func(x - xi)
    gvals /= np.trapz(gvals, x)
    G[i, :] = gvals * dx
# Solve  $(G^T G + 2 D^T D) m = G^T d$  with  $d = \ln(u_s\_noisy)$ 
GTG = G.T.dot(G)
DTD = D.T.dot(D)
lhs = GTG + (lam**2) * DTD #LHS and RHS of the linearized model
rhs = G.T.dot(d)

# Solution of the inverse model
u_b_est = solve(lhs, rhs)

# Surface velocity obtained from the estimated basal velocity put in the
↳forward model
ln_us_est = G.dot(np.log(1.0 + u_b_est))
u_s_est = np.exp(ln_us_est)
plt.figure(figsize=(10,5))
plt.plot(x, u_b, 'k--', lw=2, label='Input basal velocity in FM')#FM- forward
↳model
plt.plot(x, u_b_est, 'g-', lw=1.5, label='')
plt.xlabel('x'); plt.ylabel('Basal velocity'); plt.title('Input basal
↳velocity(FM) and estimated basal velocity(IM)')#IM-inverse model
plt.legend(); plt.grid(alpha=0.25); plt.xlim(0,100)
plt.show()

plt.figure(figsize=(10,5))
plt.plot(x, u_s, 'k--', lw=2, label='calculated surface velocity (FM)')

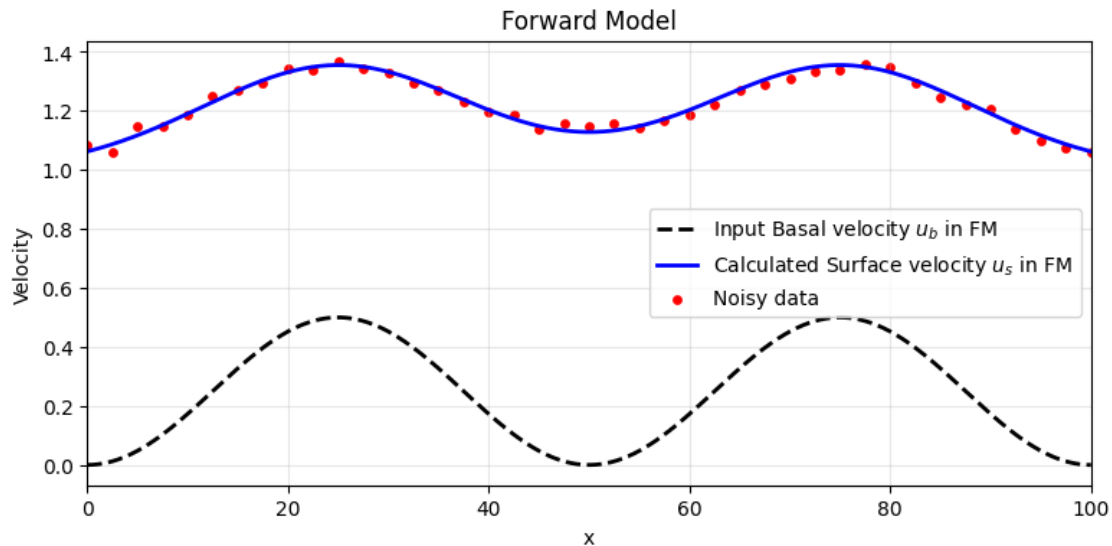
```

```
plt.plot(x, u_s_noisy, 'r.', markersize=2, label='Noisy surface velocity (FM)')
plt.plot(x, u_s_est, 'b-', lw=1.5, label='surface velocity from estimated_
↳basal')
plt.xlabel('x'); plt.ylabel('Surface velocity'); plt.title('Input surface_
↳velocity(IM) and estimated surface velocity(FM)')
plt.legend(); plt.grid(alpha=0.25); plt.xlim(0,100)
plt.show()
```

C:\Users\aparn\AppData\Local\Temp\ipykernel_5768\1982776831.py:16:

DeprecationWarning: `trapz` is deprecated. Use `trapezoid` instead, or one of the numerical integration functions in `scipy.integrate`.

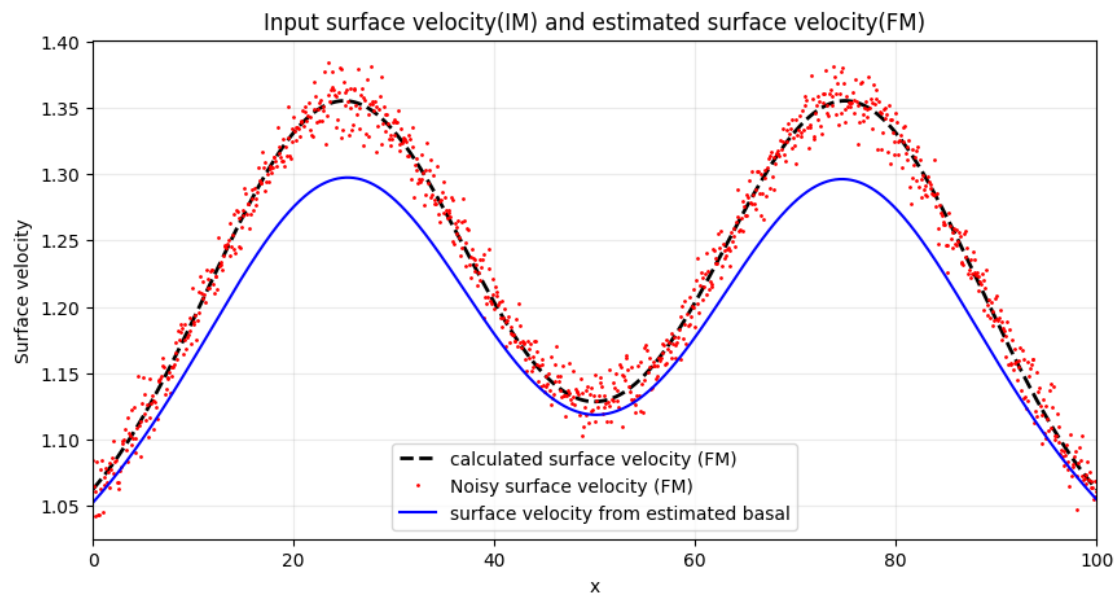
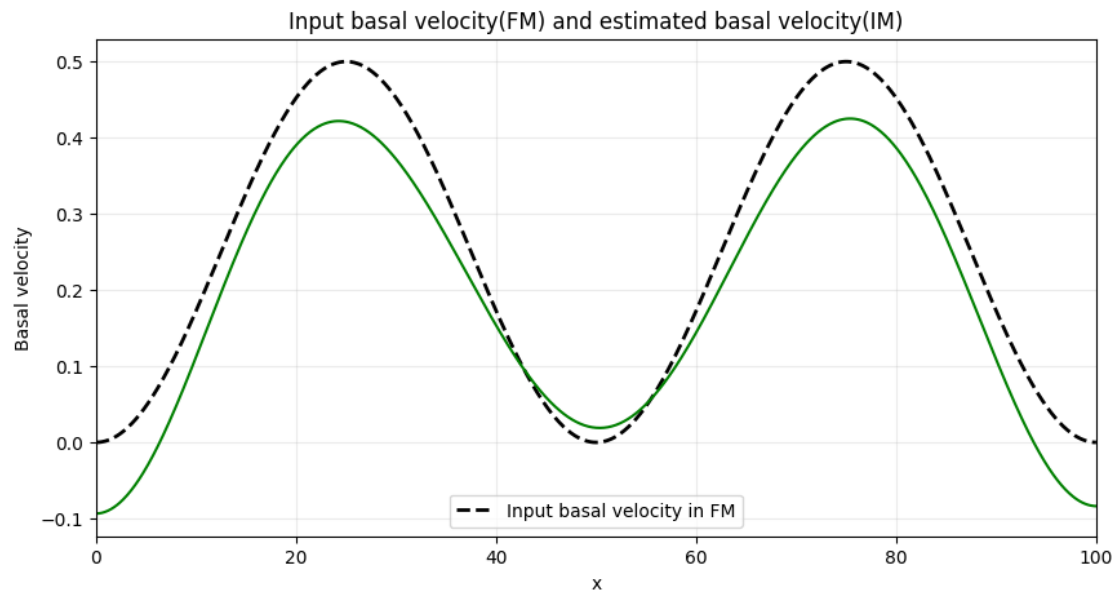
```
g /= np.trapz(g, x_kernel)
```



C:\Users\aparn\AppData\Local\Temp\ipykernel_5768\1982776831.py:51:

DeprecationWarning: `trapz` is deprecated. Use `trapezoid` instead, or one of the numerical integration functions in `scipy.integrate`.

```
gvals /= np.trapz(gvals, x)
```



[]: